

Paylaştığınız Python kodu, az önce matematiksel olarak çözdüğümüz sorunun **kaba kuvvet (brute force) yöntemiyle tüm ihtimalleri deneyerek** doğru sonucu bulmasını sağlayan bir algoritmadır. Code içinde bir de MIT Lisansı bildisiimi yer almaktadır.

Kodun adım adım mantığı şu şekildedir:

1. Lisans Metni (MIT License)

Kodun en başında yer alan üç tırnak `"""` içerisindeki metin, yazılımın **ücretsiz, değiştirilebilir ve dağıtılabilir** olduğunu belirten yasal bir MIT lisansıdır. Kodun çalışmasına doğrudan bir etkisi yoktur.

2. Fonksiyonun Tanımlanması ve Değişkenler

```
def solve():  
    digits = list(range(10))  
    min_val = float('inf')  
    best_abc = None
```

- **digits = list(range(10))**: 0'dan 9'a kadar olan rakamları içeren bir liste oluşturur: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9].
- **min_val = float('inf')**: Bulunacak en küçük değeri hafızada tutacak değişkendir. Karşılaştırma yaparken ilk döngüde her halükarda güncellenmesi için başlangıçta **sonsuz büyük değer** (inf) olarak atanmıştır.
- **best_abc = None**: En küçük sonucu veren (a, b, c) üçlüsünü kaydetmek için boş bir değişken oluşturur.

3. İç İç Döngüler (Tüm İhtimalleri Deneme)

```
for a in digits:  
    for b in digits:  
        if b == a: continue  
        for c in digits:  
            if c == a or c == b: continue
```

- **Üçlü for döngüsü**: a, b ve c değişkenlerinin her biri için 0'dan 9'a kadar olan tüm rakamları sırayla dener.
- **if b == a: continue**: Sorudaki **"birbirinden farklı"** şartını sağlamak içindir. Eğer b rakamı a ile aynıysa, altındaki işlemleri geçerek bir sonraki b değerine atlar.
- **if c == a or c == b: continue**: Benzer şekilde c rakamı, a veya b ile aynıysa o adımı atlar. Böylece sadece üçünün de farklı olduğu durumlar incelenir.

4. Matematiksel Hesaplama ve Karşılaştırma

```
val = a - 2*b + 3*c
if val < min_val:
    min_val = val
    best_abc = (a, b, c)
```

- **val = a - 2*b + 3*c**: Geçerli a, b, c rakamları için matematiksel ifadeyi hesaplar.
- **if val < min_val**: Eğer hesaplanan bu yeni değer (val), o ana kadar bulunmuş en küçük değerden (min_val) daha küçükse:
 - min_val değişkenini bu yeni küçük değerle günceller.
 - Bu değeri veren (a, b, c) rakamlarını best_abc içerisine kaydeder.

5. Sonucu Yazdırma ve Çalıştırma

```
print(f"Min value: {min_val}, a={best_abc[0]}, b={best_abc[1]}, c={best_abc[2]}")
```

```
solve()
```

- **print(...)**: Tüm ihtimaller (toplamda $10 \times 9 \times 8 = 720$ farklı durum) denendikten sonra ulaşılan en küçük değeri ve bunu sağlayan rakamları ekrana yazdırır.
- **solve()**: Yukarıda tanımlanan tüm bu süreci başlatan fonksiyon çağrısıdır.

Kod çalıştırıldığında ekrana tam olarak matematiksel analizimizle eşleşen şu çıktıyı verecektir:

Min value: -17, a=1, b=9, c=0

Bu kod üzerinde herhangi bir değişiklik yapmak ister misiniz? Örneğin:

- Kodun **çalışma hızını (performansını)** artıracak itertools.permutations kullanımını göstermemi ister misiniz?
- Aynı mantıkla ifadenin **en büyük (maksimum) değerini** bulacak şekilde kodu güncelleyelim mi?